



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

# Programação Dinâmica 1

- ❖ O que é
- ❖ Problemas clássicos
- ❖ Como reconstruir a resposta

Omena <jvrco>

Leonardo <las4>

Thawan <trs3>



Maratona **CIn**



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

# O que é DP?

Backtracking

- Otimizações

- + Memoização

# Backtracking



Maratona CIn



Centro de  
Informática  
UFPE



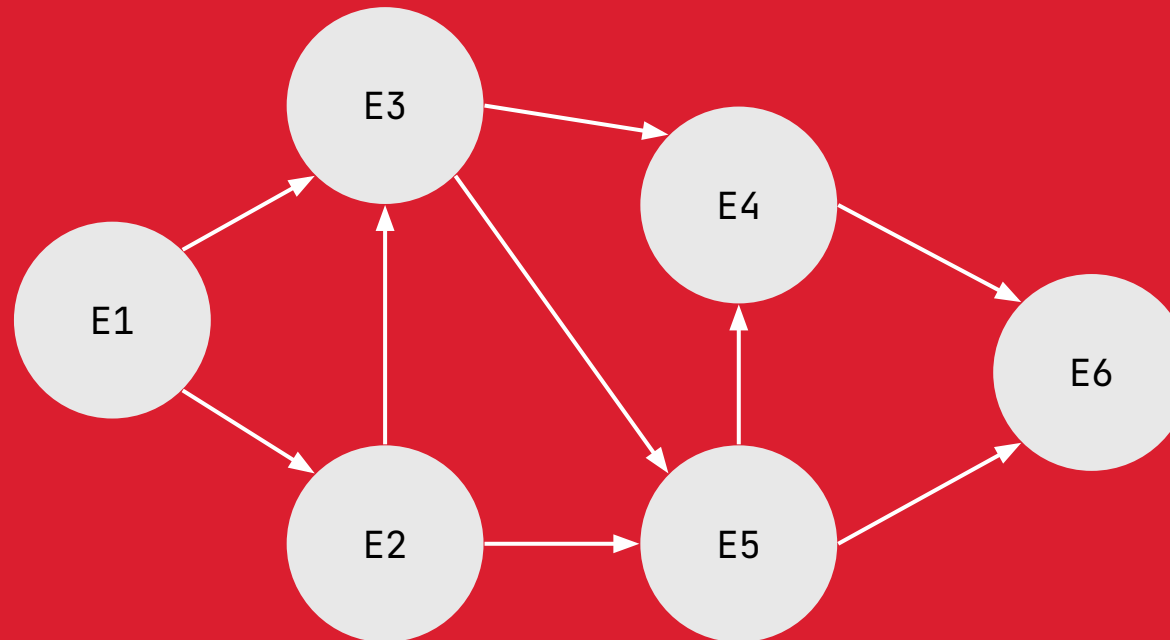
UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Estado

- Informação mínima para resolver o subproblema

## Transição

- Escolha que te leva para outro estado



# Problema



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

Dado um array  $A$  de tamanho  $N$ , encontre a maior soma possível de valores de uma subsequência de  $A$  em que cada par de elementos consecutivos tenha diferença absoluta menor ou igual a  $K$ .

- $N \leq 10^5$
- $0 \leq A[i], K \leq 10$

Exemplos:

- $A = [8, 10, 6, 4, 2, 4]$  |  $K = 2 \Rightarrow [8, 6, 4, 2, 4] \Rightarrow 24$
- $A = [5, 2, 4, 3, 1]$ , |  $K = 3 \Rightarrow [5, 2, 4, 3, 1] \Rightarrow 15$
- $A = [1, 4, 5, 6, 4, 3, 4, 2, 5]$  |  $K = 1 \Rightarrow [4, 5, 6, 5] \Rightarrow 20$

# Solucionando ...



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- Vamos montar a subsequência ótima
- Cada elemento de A pode estar ou não estar na subsequência final
- Se o último elemento da subsequência atual tem valor X:
  - Se  $|A[i] - X| \leq K$  e eu decido que A[i] vai estar na subsequência final, o próximo elemento, se houver, deve ser um A[j], tal que:
    - $j > i$
    - $|A[j] - A[i]| \leq K$
  - Se eu decido que A[i] não vai estar na subsequência final, o próximo elemento, se houver, deve ser um A[j], tal que:
    - $j > i$
    - $|A[j] - X| \leq K$

# Solucionando ...



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- Vamos assumir que  $A[0]$  é o primeiro elemento da subsequência final
- Eu quero saber a **subsequência de  $A[1:]$  com maior soma e valores consecutivos com diferença menor ou igual a  $K$  e primeiro elemento com distância menor ou igual a  $K$  de  $A[0]$**
- Se eu decido não colocar  $A[1]$  na subsequência:
  - Eu quero saber a **subsequência de  $A[2:]$  com maior soma e valores consecutivos com diferença menor ou igual a  $K$  e primeiro elemento com distância menor ou igual a  $K$  de  $A[0]$**
- Se  $|A[1] - A[0]| \leq K$  e eu decido colocar  $A[1]$  na subsequência:
  - Eu quero saber a **subsequência de  $A[2:]$  com maior soma e valores consecutivos com diferença menor ou igual a  $K$  e primeiro elemento com distância menor ou igual a  $K$  de  $A[1]$**

# Solucionando ...



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

Eu quero saber a **subsequência de <sufixo de A>** com maior soma e valores consecutivos com diferença menor ou igual a K e **primeiro elemento com distância menor ou igual a K de <elemento de A>**

# Solucionando ...



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

**Decidir se  $A[i]$  vai estar ou não na subsequência reduz o problema!**



# Backtracking



Maratona CIn



Centro de  
Informática  
UFPE



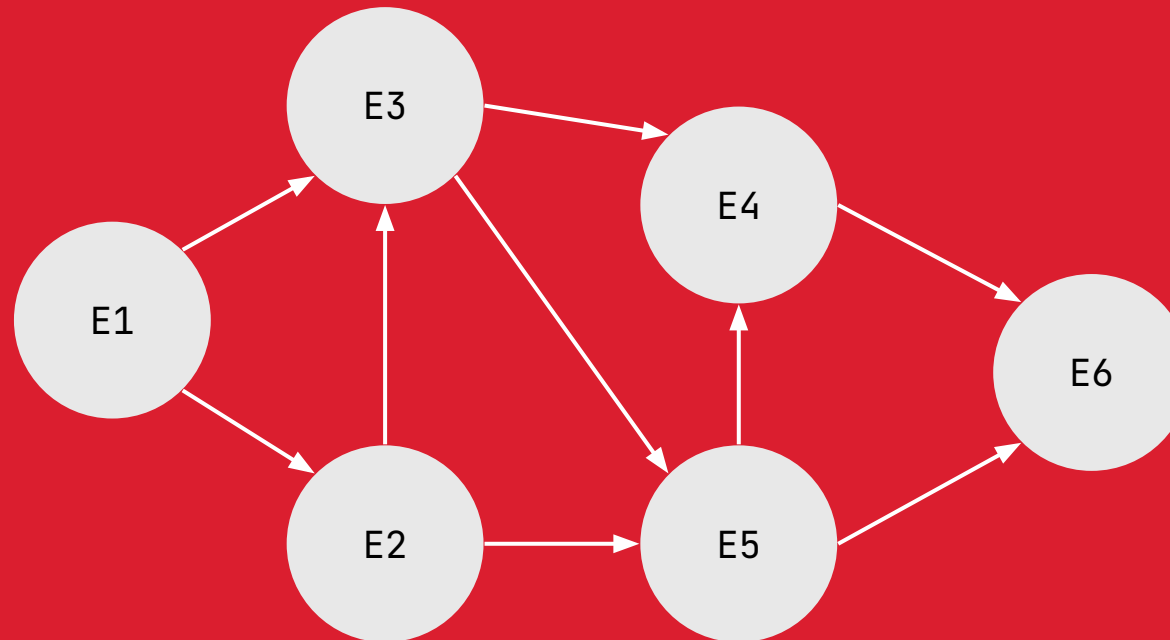
UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Estado

- Informação mínima para resolver o subproblema

## Transição

- Escolha que te leva para outro estado



# Backtracking



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Estado

$(i, X)$

- $i \Rightarrow$  Sufixo  $A[i:]$
- $X \Rightarrow$  último elemento da subsequência atual

## Transição

Selecionar ou não  $A[i]$

- $(i, X) \Rightarrow (i + 1, X)$
- $(i, X) \Rightarrow (i + 1, A[i])$

# Backtracking



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

```
fun maior_soma(i, X):  
    if i ≥ N: return 0  
  
    ans = maior_soma(i + 1, X)  
    if |A[i] - X| ≤ K:  
        nans = maior_soma(i + 1, A[i]) + A[i]  
        ans = max(ans, nans)  
  
    return ans  
  
final_ans = max0 ≤ i < N(maior_soma(i + 1, A[i]) + A[i])
```

# Backtracking



MaratonaCIn

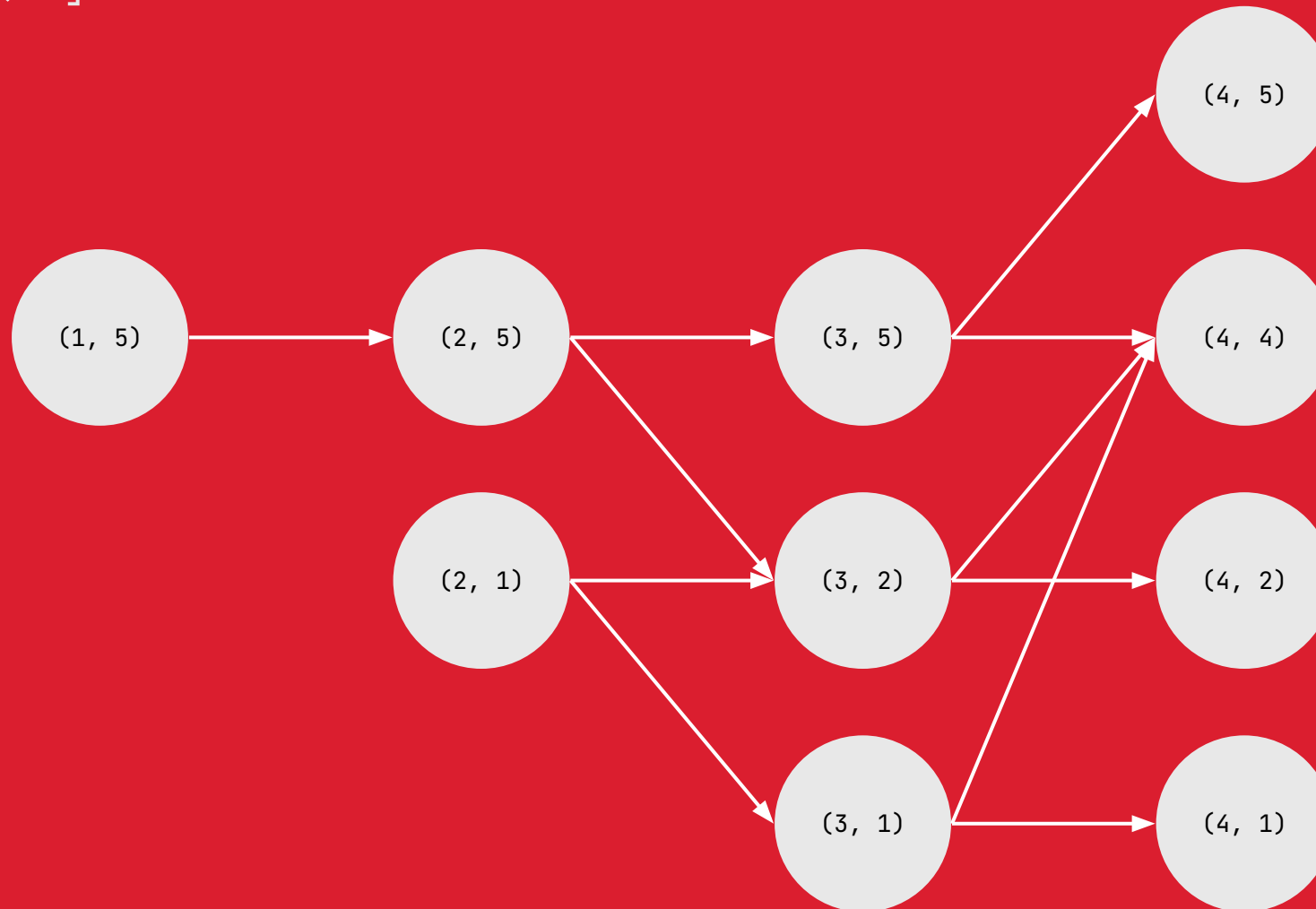


Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- $A = [5, 1, 2, 4]$
- $K = 3$



# Backtracking



MaratonaCIn

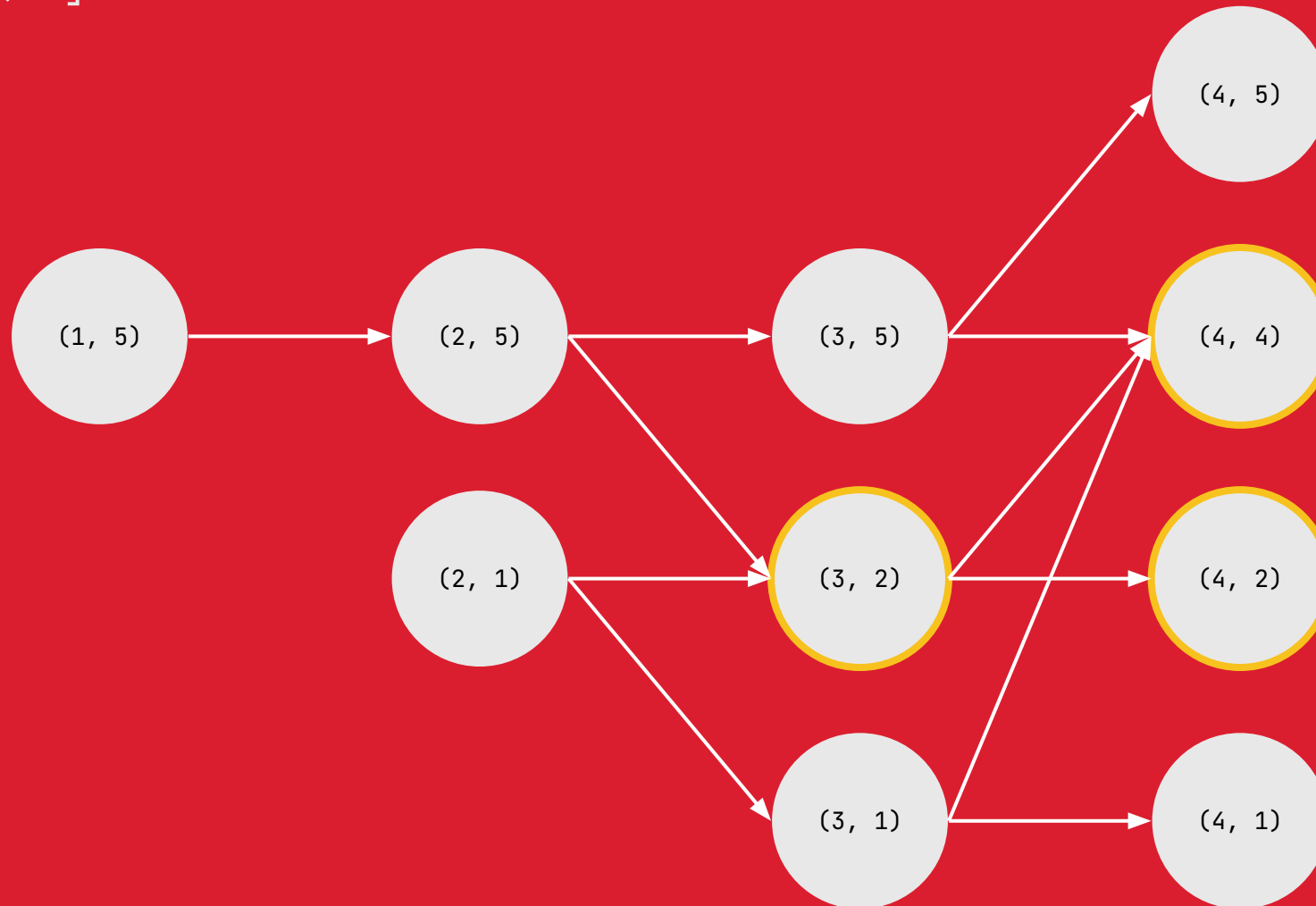


Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- $A = [5, 1, 2, 4]$
- $K = 3$



# Backtracking



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- Calculamos várias vezes a respostas para alguns estados
- O estado tem informação suficiente para resolver o subproblema, então a resposta é sempre a mesma, independente de como chegou lá

# Backtracking



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- Calculamos várias vezes a respostas para alguns estados
- O estado tem informação suficiente para resolver o subproblema, então a resposta é sempre a mesma, independente de como chegou lá

**E se salvarmos os resultados em memória?**

# Memoização



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

```
fun maior_soma(i, X):  
    if i ≥ N: return 0  
    if memo[i, X] ≠ null: return memo[i, X]  
  
    memo[i, X] = maior_soma(i + 1, X)  
    if |A[i] - X| ≤ K:  
        nans = maior_soma(i + 1, A[i]) + A[i]  
        memo[i, X] = max(memo[i, X], nans)  
  
    return memo[i, X]  
  
final_ans = max0 ≤ i < N(maior_soma(i + 1, A[i]) + A[i])
```



# DP



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- Computa todas as possibilidades de forma inteligente
- Salva as respostas de computações intermediárias
- Complexidade:
  - Memória:  $O(\text{número de estados})$
  - Tempo:  $O(\text{número de estados} * \text{número de transições por estado})$



Maratona **CIn**



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

# Problemas Clássicos



não grita



# LIS

# LIS



## O que é o LIS (Longest Increasing Subsequence)?

## O que é o LIS (Longest Increasing Subsequence)?

Dado um array, queremos achar o tamanho da maior subsequência estritamente crescente.

**Importante:** subsequência  $\neq$  subarray (podemos pular elementos).

**Exemplo:**

Array = [10, 9, 2, 5, 3, 7, 101, 9, 20]

**Pergunta:** qual o tamanho do LIS?

## O que é o LIS (Longest Increasing Subsequence)?

Dado um array, queremos achar o tamanho da maior subsequência estritamente crescente.

**Importante:** subsequência  $\neq$  subarray (podemos pular elementos).

### Exemplo:

Array = [10, 9, 2, 5, 3, 7, 101, 9, 20]

**Pergunta:** qual o tamanho do LIS?

LIS = [2, 3, 7, 9, 20]: tamanho = 5.

## Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar.

## Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar.  $O(2^n)$



## Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar.  $O(2^n)$

## Com programação dinâmica:

$dp[i]$  = tamanho da maior subsequência crescente que termina em  $i$

## Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar.  $O(2^n)$

## Com programação dinâmica:

$dp[i]$  = tamanho da maior subsequência crescente que termina em  $i$   $O(n^2)$

Com programação dinâmica:

$dp[i]$  = tamanho da maior subsequência crescente que termina em  $i$   $O(n^2)$

```

1  int solve() {
2      int n = 9;
3      vector<int> a = {10, 9, 2, 5, 3, 7, 101, 9, 20};
4
5      int answer = 0;
6      vector<int> dp(n);
7      for (int j = 0; j < n; ++j) {
8          dp[j] = 1;
9          for (int i = j - 1; i >= 0; --i) {
10             if (a[i] < a[j]) {
11                 dp[j] = max(dp[j], dp[i] + 1);
12             }
13         }
14         answer = max(answer, dp[j]);
15     }
16     return answer;
17 }
```

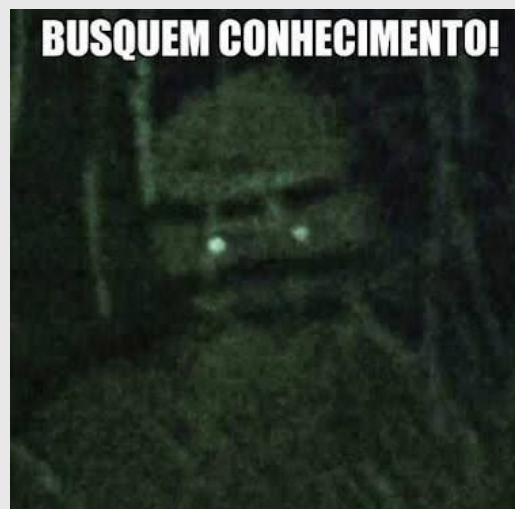
## Método ingênuo:

Para cada número (maior que o anterior escolhido), você tem duas escolhas: usar ou não usar.  $O(2^n)$

## Com programação dinâmica:

$dp[i]$  = tamanho da maior subsequência crescente que termina em  $i$   $O(n^2)$

Otimização:  $O(n * \log n)$



# LCS

# LCS



## O que é o LCS (Longest Common Subsequence)?

## O que é o LCS (Longest Common Subsequence)?

LCS = maior subsequência comum entre duas strings, preservando a ordem, mas não necessariamente contígua.

### Exemplo:

A = "acbd"

B = "abed"

**Pergunta:** qual o tamanho do LCS?

## O que é o LCS (Longest Common Subsequence)?

LCS = maior subsequência comum entre duas strings, preservando a ordem, mas não necessariamente contígua.

### Exemplo:

A = "acbd"

B = "abed"

**Pergunta:** qual o tamanho do LCS?

LIS = "abd": tamanho = 3.



## Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra.

## Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra.  $O(2^n)$

## Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra.  $O(2^n)$

## Com programação dinâmica:

$dp[i][j]$  = LCS entre  $A[0..i]$  e  $B[0..j]$

## Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra.  $O(2^n)$

## Com programação dinâmica:

$dp[i][j]$  = LCS entre  $A[0..i]$  e  $B[0..j]$

Se  $A[i] == B[j]$ :

$$dp[i][j] = dp[i - 1][j - 1] + 1$$

Senão:

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

## Método ingênuo:

Gerar todas as subsequências de uma string e verificar quais também aparecem em ordem na outra.  $O(2^n)$

**Com programação dinâmica:**  $O(n * m)$

$dp[i][j]$  = LCS entre  $A[0..i]$  e  $B[0..j]$

**Se**  $A[i] == B[j]$ :

$$dp[i][j] = dp[i - 1][j - 1] + 1$$

**Senão:**

$$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$$

# LCS

```
1  vector<vector<int>> dp;
2
3  string a;
4  string b;
5
6  int best(int i, int j) {
7      if (i < 0 or j < 0) return 0;
8      if (dp[i][j] != -1) return dp[i][j];
9
10     dp[i][j] = 0;
11     if (a[i] == b[j]) {
12         dp[i][j] = 1 + best(i - 1, j - 1);
13     } else {
14         dp[i][j] = max(best(i - 1, j), best(i, j - 1));
15     }
16     return dp[i][j];
17 }
18
19 int solve() {
20     a = "acbd";
21     int n = 4;
22     b = "abed";
23     int m = 4;
24
25     dp = vector<vector<int>>(n, vector<int>(m, -1));
26
27     return best(n - 1, m - 1);
28 }
```

# Subset Sum

# Subset Sum

O que é o Subset Sum?



Maratona**CIn**



**Centro de  
Informática**  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO



# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## O que é o Subset Sum?

É o problema de decidir se existe algum subconjunto de um conjunto de inteiros que some exatamente um valor-alvo  $T$ .

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## O que é o Subset Sum?

É o problema de decidir se existe algum subconjunto de um conjunto de inteiros que some exatamente um valor-alvo  $T$ .

## Exemplo simples:

Conjunto =  $\{3, 4, 7, 2\}$

$T = 9$

**Pergunta:** existe subconjunto que soma 9?

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## O que é o Subset Sum?

É o problema de decidir se existe algum subconjunto de um conjunto de inteiros que some exatamente um valor-alvo  $T$ .

## Exemplo simples:

Conjunto =  $\{3, 4, 7, 2\}$

$T = 9$

**Pergunta:** existe subconjunto que soma 9?

Sim:  $\{7, 2\}$  ou  $\{3, 4, 2\}$

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.  $O(2^n)$

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][s]$  = é possível fazer soma  $s$  usando os primeiros  $i$  números?

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][s]$  = é possível fazer soma  $s$  usando os primeiros  $i$  números?  $O(i * s)$

# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][s]$  = é possível fazer soma  $s$  usando os primeiros  $i$  números?  $O(i * s)$

## Com programação dinâmica (1D):

$dp[s]$  = consigo fazer soma  $s$ ?



# Subset Sum



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

Para cada número, você tem duas escolhas: usar ou não usar.

E essas escolhas influenciam a soma atual.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][s]$  = é possível fazer soma  $s$  usando os primeiros  $i$  números?  $O(i * s)$

## Com programação dinâmica (1D):

$dp[s]$  = consigo fazer soma  $s$ ?  $O(i * s)$

# Subset Sum

Com programação dinâmica (1D):

$dp[s]$  = consigo fazer soma  $s$ ?  $O(i * s)$

```
1  bool solve() {
2      int t = 1;
3      vector<int> a = {3, 4, 7, 2};
4
5      vector<bool> dp(t + 1);
6      dp[0] = true;
7
8      for (int x : a) {
9          for (int i = t - x; i >= 0; --i) {
10             if (dp[i]) {
11                 dp[i + x] = true;
12             }
13         }
14     }
15     return dp[t];
16 }
```



# Knapsack



# Knapsack

O que é o Knapsack?



# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## O que é o Knapsack?

Você tem: uma mochila com capacidade  $W$ ,  
 $n$  itens, cada um com: peso  $w[i]$  e valor  $v[i]$

**Pergunta:** qual o maior valor que dá pra colocar na mochila sem passar do peso?

Cada item pode ser pego 1 vez ou não pego

# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Exemplo:

Você tem: uma mochila com capacidade  $W = 7$ ,  
3 itens, cada um com: peso  $w[i]$  e valor  $v[i]$

| Item | Peso | Valor |
|------|------|-------|
| 1    | 3    | 4     |
| 2    | 4    | 5     |
| 3    | 2    | 3     |
| 4    | 6    | 7     |

**Pergunta:** qual o maior valor que dá pra colocar na mochila sem passar do peso?

# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Exemplo:

Você tem: uma mochila com capacidade  $W = 7$ ,  
3 itens, cada um com: peso  $w[i]$  e valor  $v[i]$

| Item | Peso | Valor |
|------|------|-------|
| 1    | 3    | 4     |
| 2    | 4    | 5     |
| 3    | 2    | 3     |
| 4    | 6    | 7     |

**Pergunta:** qual o maior valor que dá pra colocar na mochila sem passar do peso? 9

# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar.



# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][w]$  = melhor valor usando itens até o  $i$ , com capacidade  $w$

# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][w]$  = melhor valor usando itens até o  $i$ , com capacidade  $w$   $O(n * W)$

## Com programação dinâmica (1D):

$dp[w]$  = melhor valor com capacidade  $w$

# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Método ingênuo:

O método ingênuo é simplesmente tentar todas as combinações possíveis de itens. Para cada item, você tem 2 escolhas: pegar (se couber) ou não pegar.  $O(2^n)$

## Com programação dinâmica (2D):

$dp[i][w]$  = melhor valor usando itens até o  $i$ , com capacidade  $w$   $O(n * W)$

## Com programação dinâmica (1D):

$dp[w]$  = melhor valor com capacidade  $w$   $O(n * W)$

# Knapsack



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

Com programação dinâmica (1D):

$dp[w]$  = melhor valor com capacidade  $w$   $O(n * W)$

```
1  int solve() {
2      int n = 4, W = 7;
3      vector<int> w = {3, 4, 2, 6};
4      vector<int> v = {4, 5, 3, 7};
5
6      vector<int> knapsack(W + 1);
7      for (int i = 0; i < n; ++i) {
8          for (int k = W - w[i]; k >= 0; --k) {
9              knapsack[k + w[i]] = max(knapsack[k + w[i]], knapsack[k] + v[i]);
10         }
11     }
12     return knapsack[W];
13 }
```



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

como  
reconstruir a  
resposta

# Como Reconstruir a Resposta



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

**O que a DP normalmente faz:**

Calcula o valor ótimo de um problema.

ex: menor custo, maior soma...

**Mas como recuperamos as decisões que levaram a esse valor?**

Muitas questões pedem qual caminho foi seguido, itens escolhidos e operações que foram feitas.

# Como Reconstruir a Resposta



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

## Caminho de maior soma em uma matriz

**Problema:** Sair de (1,1) até (3,3), somando o máximo possível e só se movimentando para baixo ou para direita

| $a[i][j]$ | $j = 1$ | $j = 2$ | $j = 3$ |
|-----------|---------|---------|---------|
| $i = 1$   | 5       | 1       | 4       |
| $i = 2$   | 2       | 7       | 2       |
| $i = 3$   | 3       | 1       | 9       |

| $dp[i][j]$ | $j = 1$ | $j = 2$ | $j = 3$   |
|------------|---------|---------|-----------|
| $i = 1$    | 5       | 6       | 10        |
| $i = 2$    | 7       | 14      | 16        |
| $i = 3$    | 10      | 15      | <b>25</b> |

$$dp[i][j] = a[i][j] + \max(dp[i-1][j], dp[i][j-1])$$

# Como Reconstruir a Resposta



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

| dp[i][j] | j = 1 | j = 2 | j = 3 |
|----------|-------|-------|-------|
| i = 1    | 5     | 6     | 10    |
| i = 2    | 7     | 14    | 16    |
| i = 3    | 10    | 15    | 25    |

Diagram illustrating the dynamic programming table (dp[i][j]) with values and arrows indicating the path reconstruction:

- From (1,1) to (2,1): vertical arrow down.
- From (2,1) to (2,2): horizontal arrow right.
- From (2,2) to (2,3): horizontal arrow right.
- From (2,1) to (3,1): vertical arrow down.
- From (3,1) to (3,2): vertical arrow down.
- From (3,2) to (3,3): vertical arrow down.

| rec[i][j] | j = 1 | j = 2 | j = 3 |
|-----------|-------|-------|-------|
| i = 1     | start | (1,1) | (1,2) |
| i = 2     | (1,1) | (2,1) | (2,2) |
| i = 3     | (2,1) | (2,2) | (2,3) |

Diagram illustrating the reconstruction table (rec[i][j]) with values and arrows indicating the path reconstruction:

- From (1,1) to (2,1): horizontal arrow left.
- From (2,1) to (2,2): horizontal arrow left.
- From (2,2) to (2,3): horizontal arrow left.
- From (2,1) to (3,1): horizontal arrow left.
- From (3,1) to (3,2): horizontal arrow left.
- From (3,2) to (3,3): horizontal arrow left.

Caminho encontrado: (1,1) → (2,1) → (2,2) → (2,3) → (3,3)



# Como atualizar as tabelas



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

- `dp[i]` - Guarda o melhor valor para aquela posição
- `rec[i]` - Guarda de qual posição anterior veio o valor

```
1  int n = 5;
2
3
4  vector<long long> dp(n, LLONG_MIN);
5  vector<int> rec(n, -1);
6
7  int start = 0;
8  dp[start] = 0;
9
10 for (int u = 0; u < n; u++) {
11     for (int v : adj[u]) {
12         long long cand = dp[u] + w[u][v];
13
14         if (cand > dp[v]) {
15             dp[v] = cand;
16             rec[v] = u;
17         }
18     }
19 }
20
```

# Como reconstruir a resposta



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

```
1  vector<int> caminho;  
2  
3  int s = estado_final;  
4  
5  while (s != estado_base) {  
6      caminho.push_back(s);  
7      s = rec[s];  
8  }  
9  
10 caminho.push_back(estado_base);  
11 reverse(caminho.begin(), caminho.end());  
12
```



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

# Dúvidas?



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO

# Dúvidas?

**Próximo passo:  
Fazer o Homework!**



MaratonaCIn



Centro de  
Informática  
UFPE



UNIVERSIDADE  
FEDERAL  
DE PERNAMBUCO



# That's all, Folks!

